



Predicting Median House Value in Boston

Author: Seif H. Kungulio

Date: February 15, 2026

A comprehensive analysis to predict median house values in Boston suburbs using various machine learning techniques.

Business Understanding

Background & Problem Statement

Boston's housing authority and urban developers face the challenge of accurately estimating property values across different neighborhoods. Property valuation depends on multiple interrelated factors—such as crime rate, accessibility to major roads, environmental quality, and local amenities—which can be difficult to model using traditional statistical techniques alone.

Business Objectives

Build a predictive model to estimate median owner-occupied home value (`medv`) using neighborhood-level indicators. The goal is to support:

- neighborhood investment planning,
- redevelopment prioritization,
- and value forecasting under changing conditions.

Success Criteria

- **Primary metric:** RMSE (lower is better)
- **Secondary metrics:** MAE and R^2
- **Practical goal:** A model that generalizes well (low validation error) and offers interpretable drivers (feature importance / coefficients).

Python Version

Below is the Python equivalent of the R workflow. It is organized as a notebook-style script so you can paste it directly into a `.ipynb`, Jupyter notebook, or a `.py` script with markdown comments.

```
# Imports
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, RepeatedKFold, GridSearchCV,
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, ElasticNet
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.inspection import permutation_importance

# Try OpenML Boston first; if unavailable, fallback to local CSV/parquet if provided
from sklearn.datasets import fetch_openml
```

Data Understanding

```
# Load dataset
boston = fetch_openml(name="boston", version=1, as_frame=True)
boston_df = boston.frame.copy()

# Ensure lowercase column names for compatibility with the R document
boston_df.columns = [c.lower() for c in boston_df.columns]

# Target handling
if "medv" not in boston_df.columns and "target" in boston_df.columns:
    boston_df = boston_df.rename(columns={"target": "medv"})

print(boston_df.info())
print(boston_df["medv"].describe())
```

Data Dictionary

The dataset contains 506 observations and 14 variables, including 13 predictors and the target `medv`.

Data Quality Assessment

```
# Missing values
print(boston_df.isna().sum())

# Duplicate rows
print("Duplicates:", boston_df.duplicated().sum())

# Quick profile
print(boston_df.describe(include="all"))
```

Exploratory Data Analysis

```
# Target distribution
plt.figure(figsize=(10, 5))
sns.histplot(boston_df["medv"], bins=30, kde=False)
plt.title("Distribution of Median Home Value (medv)")
plt.xlabel("medv ($1000s)")
plt.ylabel("Count")
plt.show()
```

```
# Relationship with key predictors
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

sns.regplot(data=boston_df, x="rm", y="medv", scatter_kws={"alpha": 0.6}, ax=axes[0,
axes[0, 0].set_title("medv vs rm (rooms)")

sns.regplot(data=boston_df, x="lstat", y="medv", scatter_kws={"alpha": 0.6}, ax=axes
axes[0, 1].set_title("medv vs lstat")

sns.scatterplot(data=boston_df, x="crim", y="medv", alpha=0.6, ax=axes[1, 0])
axes[1, 0].set_xscale("log")
axes[1, 0].set_title("medv vs crim (log scale)")

sns.regplot(data=boston_df, x="nox", y="medv", scatter_kws={"alpha": 0.6}, ax=axes[1
axes[1, 1].set_title("medv vs nox")

plt.tight_layout()
plt.show()
```

```
# Correlation heatmap
plt.figure(figsize=(12, 8))
corr = boston_df.corr(numeric_only=True)
sns.heatmap(corr, cmap="coolwarm", center=0)
plt.title("Correlation Heatmap")
plt.show()
```

Data Preparation

```
# Train/test split
X = boston_df.drop(columns=["medv"])
y = boston_df["medv"].astype(float)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=123
)

print(X_train.shape, X_test.shape)
```

```
# Numeric preprocessing
numeric_features = X.columns.tolist()

numeric_preprocessor = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_preprocessor, numeric_features)
])
```

Modeling

```
# Evaluation helpers

def rmse(actual, pred):
    return np.sqrt(mean_squared_error(actual, pred))

def mae(actual, pred):
    return mean_absolute_error(actual, pred)

def r2(actual, pred):
    return r2_score(actual, pred)

cv = RepeatedKFold(n_splits=10, n_repeats=3, random_state=123)
```

Baseline Model

```
baseline_pred = np.repeat(y_train.mean(), len(y_test))

baseline_metrics = pd.DataFrame({
    "Model": ["Baseline (Train Mean)"],
    "RMSE": [rmse(y_test, baseline_pred)],
    "MAE": [mae(y_test, baseline_pred)],
    "R2": [np.nan]
})
```

```
baseline_metrics
```

Multiple Linear Regression

```
lm_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LinearRegression())
])

lm_pipeline.fit(X_train, y_train)
lm_pred = lm_pipeline.predict(X_test)

lm_metrics = pd.DataFrame({
    "Model": ["Linear Regression"],
    "RMSE": [rmse(y_test, lm_pred)],
    "MAE": [mae(y_test, lm_pred)],
    "R2": [r2(y_test, lm_pred)]
})

lm_metrics
```

```
# Diagnostics
res = y_test - lm_pred

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.scatterplot(x=lm_pred, y=res, alpha=0.6, ax=axes[0])
axes[0].axhline(0, color="black", linestyle="--")
axes[0].set_title("Residuals vs Predictions")
axes[0].set_xlabel("Predicted medv")
axes[0].set_ylabel("Residual")

import scipy.stats as stats
stats.probplot(res, dist="norm", plot=axes[1])
axes[1].set_title("Residual QQ Plot")

plt.tight_layout()
plt.show()
```

Elastic Net

```
elastic_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", ElasticNet(max_iter=10000))
])

param_grid = {
    "model__alpha": np.logspace(-3, 1, 40),
    "model__l1_ratio": [0.0, 0.25, 0.5, 0.75, 1.0]
}
```

```

glmnet_search = GridSearchCV(
    elastic_pipeline,
    param_grid=param_grid,
    scoring="neg_root_mean_squared_error",
    cv=cv,
    n_jobs=-1
)

glmnet_search.fit(X_train, y_train)
glmnet_pred = glmnet_search.predict(X_test)

glmnet_metrics = pd.DataFrame({
    "Model": ["Elastic Net"],
    "RMSE": [rmse(y_test, glmnet_pred)],
    "MAE": [mae(y_test, glmnet_pred)],
    "R2": [r2(y_test, glmnet_pred)]
})

print(glmnet_search.best_params_)
glmnet_metrics

```

```

# Elastic Net coefficients
best_model = glmnet_search.best_estimator_.named_steps["model"]
feature_names = X.columns
coef_df = pd.DataFrame({
    "term": feature_names,
    "estimate": best_model.coef_
}).sort_values(by="estimate", key=np.abs, ascending=False)

coef_df.head(12)

```

Random Forest

```

rf_model = RandomForestRegressor(random_state=42)

rf_param_grid = {
    "n_estimators": [300, 500],
    "max_features": [3, 4, 5, 6, 7, 8],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

rf_search = GridSearchCV(
    rf_model,
    param_grid=rf_param_grid,
    scoring="neg_root_mean_squared_error",
    cv=cv,
    n_jobs=-1
)

rf_search.fit(X_train, y_train)
rf_pred = rf_search.predict(X_test)

```

```

rf_metrics = pd.DataFrame({
    "Model": ["Random Forest"],
    "RMSE": [rmse(y_test, rf_pred)],
    "MAE": [mae(y_test, rf_pred)],
    "R2": [r2(y_test, rf_pred)]
})

print(rf_search.best_params_)
rf_metrics

```

```

# Random Forest feature importance
rf_best = rf_search.best_estimator_
imp_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_best.feature_importances_
}).sort_values("Importance", ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(data=imp_df.head(12), x="Importance", y="Feature", orient="h")
plt.title("Top Feature Importances (Random Forest)")
plt.show()

```

Evaluation

```

results = pd.concat([
    baseline_metrics,
    lm_metrics,
    glmnet_metrics,
    rf_metrics
], ignore_index=True).sort_values("RMSE")

results

```

```

# Prediction quality plot for best model
best_model_name = results.iloc[0]
<span style="display:none">[^2]</span>

<div align="center">✖</div>

[^1]: medv_boston.pdf
[^2]: medv_boston.Rmd

```